

Trabalho realizado por (PM.05.2025175346):

- Miguel Marcelo, 2025175346
- Tomás Mateus, 2025180241
- Rodrigo Coelho, 2025177273

1. Descrição do Jogo: "Space Invaders: Evolution"

- **Objetivo:** O jogador controla uma nave defensora que deve destruir hordas de invasores alienígenas antes que estes atinjam a base ou destruam a nave do jogador. O jogador deve pressionar o "spacebar" para disparar lasers contra os alienígenas para que eles desapareçam. O jogador deve tentar sobreviver durante um determinado número de rondas.
- **Regras Principais:** O jogador move-se lateralmente (teclas 'A' e 'D') e dispara ('SPACEBAR'); os inimigos movem-se em bloco, descendo de nível ao atingir as bordas. O jogo termina se o jogador perder todas as vidas, deixando os inimigos chegar à linha de base 3 vezes. O jogador ganha se conseguir passar pelo número de rondas necessário.
- **Diferenciação:** O jogo inclui diferentes tipos de inimigos com comportamentos distintos e *power-ups* que caem aleatoriamente para obter mais pontos. O jogo tem um nível fácil, um nível difícil (com mais rondas e com os inimigos mais rápidos de combater) e um nível "endless" em que o jogador tenta sobreviver durante o máximo de rondas possível. O menu disponibiliza os níveis, ficando com um "checkmark" sempre que um dos dois níveis é completado, e o jogador consegue ver a sua pontuação máxima no modo endless.

2. Modelação do Domínio e Classes

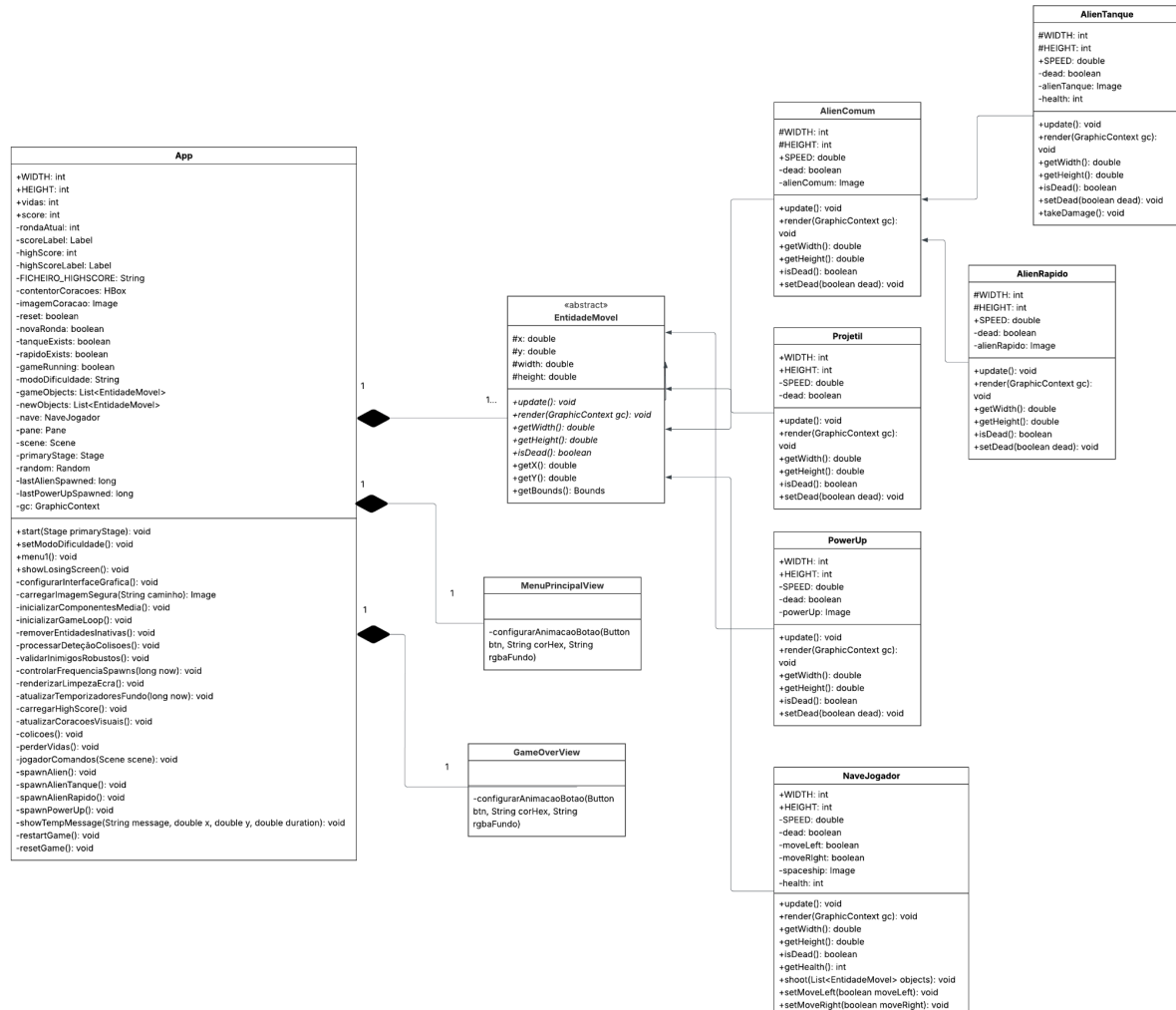
Para cumprir o requisito de **herança e polimorfismo**, a estrutura de classes será organizada da seguinte forma:

Hierarquia de Entidades (Exemplo de UML)

- **EntidadeMovel (Classe Abstrata)**: Contém atributos como **x**, **y**, **width** e **height**. Define um conjunto de métodos abstratos, que cada uma das subclasses irá implementar. Eis as subclasses:
 - **NaveJogador**: Gere a entrada do teclado e dispara projéteis.
 - **AlienComum**: Representa o inimigo. Tem uma vida e uma velocidade não tão elevada. Define a base para todos os invasores.
 - **AlienTanque**: Requer dois disparos para ser destruído. Move-se mais devagar;
 - **AlienRapido**: Move-se a uma velocidade superior. Tem o mesmo número de vidas que o alien comum.
 - **Projétil**: Gere a colisão e o dano causado aos inimigos. Representa os “tiros” da nave do jogador;
 - **PowerUp**: Representa um power-up. Quando o projétil atinge um, o jogador ganha um número elevado de pontos;
- **App (Lógica do Jogo)**: Controla a pontuação, o estado da vaga (vitória/derrota) e a separação da interface. Define, ainda mais, mais duas “scenes”
 - **MenuPrincipalView**: Representa o menu do jogo, com a seleção de níveis;
 - **GameOverView**: Representa o ecrã quando o jogador perde o jogo;

3. Modelo de Classes

Com o que foi descrito na segunda parte, e com o número limitado de métodos e atributos que neste momento dão para serem implementados, criou-se este diagrama de UML:



4. Arquitetura da Aplicação

Modelo de Domínio

Como foi explicado previamente, EntidadeMovel é a classe mais importante, pois ela irá incorporar todas as entidades do jogo. Ou seja, a nave do jogador, os inimigos, os power-ups e até os projéteis são herdados da classe abstrata EntidadeMovel. Esta classe tem como atributos (**encapsulamento**):

- **double x**: Determina a posição horizontal do objeto no jogo;
- **double y**: Determina a posição vertical do objeto no jogo;
- **double WIDTH**: Determina a largura do objeto (pré-definido em cada objeto);
- **double HEIGHT**: Determina a altura do objeto (pré-definido em cada objeto);

Para todos os objetos, são implementados estes métodos, que são métodos abstratos da classe EntidadeMovel (herança e polimorfismo - os objetos das subclasses devem substituir os objetos da EntidadeMovel sem alterar o estado do programa):

- **public abstract void update()**: Permite atualizar a posição do objeto no jogo;
- **public abstract void render(GraphicsContext gc)**: Permite fazer com que os objetos sejam apresentados com imagens. Caso as imagens não sejam introduzidas, elas substituídas com retângulos;
- **public abstract boolean isDead()**: Booleano que permite determinar se o objeto está “morto” ou não;
- **public abstract double getWidth()** E **public abstract double getHeight()**: Getters para a largura e a altura. É preciso ter em consideração que cada objeto usa larguras e alturas diferentes.

Na classe EntidadeMovel, é também estabelecido getters para x e y, mas também é implementado um método “public Bounds getBounds()” que permite introduzir os próprios objetos no jogo. Eis algumas notas:

- A classe NaveJogador usa booleans “moveLeft” e “moveRight” (com setters) que, mais logo, vão permitir o jogador mudar de posições com a nave;
- A classe AlienComum herda as classes AlienTanque e AlienRapido (herança e polimorfismo - os objetos das subclasses devem substituir os objetos da EntidadeMovel sem alterar o estado do programa);
- A classe AlienTanque tem um método “public void takeDamage()” que permite decrementar o número de vidas do alien tanque. Caso as vidas chegam a 0, o booleano dead é mudado para true;

Controlo da Aplicação

Aqui vem a classe mais desafiadora da implementação do jogo: App, juntamente com GameOverView e MenuPrincipalView. É aqui onde será feito o controle da aplicação. Para isso, é preciso implementar um número elevado de atributos (encapsulamento):

- **public static final int WIDTH:** Define a largura da janela do jogo;
- **public static final int HEIGHT:** Define a altura da janela do jogo;
- **public static int vidas:** Define o número de vidas do jogador;
- **private int rondaAtual:** Define o número de rondas. O jogador chega a uma nova ronda a cada 500 pontos;
- **private final HBox contentorCoracoes E private Image imagemCoracao:** Elementos visuais para o Sistema de Corações (vidas);
- **private int score:** Define a pontuação do jogador;
- **private final Label scoreLabel:** Define o Label que apresenta a pontuação do jogador;
- **private int highScore:** Define a pontuação máxima do jogador no modo endless;
- **private final Label highScoreLabel:** Define o Label que apresenta a pontuação máxima do jogador no modo endless;
- **private static final String FICHEIRO_HIGHSCORE:** Ficheiro em que será guardada a pontuação máxima do jogador no modo endless;
- **private boolean reset:** Booleano que serve para indicar se o jogo deve reiniciar ou não;
- **private boolean novaRonda:** Booleano que serve para indicar se a mensagem que anuncia uma nova ronda apareceu ou não
- **private boolean tanqueExists:** Booleano que serve para indicar se um alien tanque existe ou não;
- **private String modoDificuldade :** Serve para indicar o nível (pode ser "FACIL", "DIFICIL" ou, por definição, "ENDLESS");
- **private boolean gameRunning:** Booleano que serve para indicar se o jogo está a decorrer ou não;
- **private final List<EntidadeMovel> gameObjects:** Lista de entidades móveis;
- **private final List<EntidadeMovel> newObjects:** Lista de entidades móveis com o propósito de adicionar entidades novas;
- **private NaveJogador nave:** Representa a nave do jogador;
- **private Pane pane:** Node de raiz para o jogo;
- **private Scene scene:** Scene do jogo;
- **private Stage primaryStage:** Stage da aplicação. Ao ser colocada assim, vai facilitar a mudança de ecrãs;
- **private final Random random:** Gerador de números aleatórios;
- **private long lastAlienSpawned E private long lastPowerUpSpawned:** Servem para controlar os "spawns" de inimigos e power-ups;
- **private GraphicsContext gc:** Permite carregar imagens ao programa;

Estes atributos serão usados para implementar muitos, muitos mas muitos métodos que serão necessários para pôr o jogo a funcionar:

- Existem métodos para começar ou parar o jogo;
- Existem métodos para carregar informação (imagens, pontuação, aliens, power-ups, etc);
- Existem métodos para definir o ecrã do menu principal e o ecrã de fim de jogo, utilizando implementações básicas de criação de cenários do JavaFX. Existem ainda métodos para melhorar a visualização de certos botões ou labels;
- Existem métodos para guardar e carregar a pontuação máxima no modo endless. Sempre que se quer implementar cada um destes objetos, é preciso adicioná-lo à lista gameObjects;
- Existe um método que permite definir os comandos do jogador, usando “scene.setOnKeyPressed” (quando um botão é apertado) e “scene.setOnKeyReleased” (quando esse botão deixa de ser apertado). Para manipular isto tudo, é preciso fazer uso dos setters “setMoveLeft/Right” definidos na classe NaveJogador;
- Existe um método que permite que, sempre que um alien passar para demasiado baixo do ecrã, o jogador perde uma vida;
- Existe um método para gerir colisões entre objetos, principalmente quando um projétil atinge um power-up ou um inimigo;

Por fim, chega o método “start(Stage primaryStage)”, que irá fazer a implementação completa. Dentro de um AnimationTimer, e enquanto gameRunning for igual a true, são introduzidos todos os tipos de aliens, power-ups, e são chamados os métodos para definir os comandos do utilizador, a gestão de vidas e as colisões. É também feita, num ciclo for, a atualização de cada objeto no gameObjects que foi recolhida ao longo do tempo. Usando um Iterator, é possível percorrer essa lista e remover os objetos cuja booleano dead é igual a true.

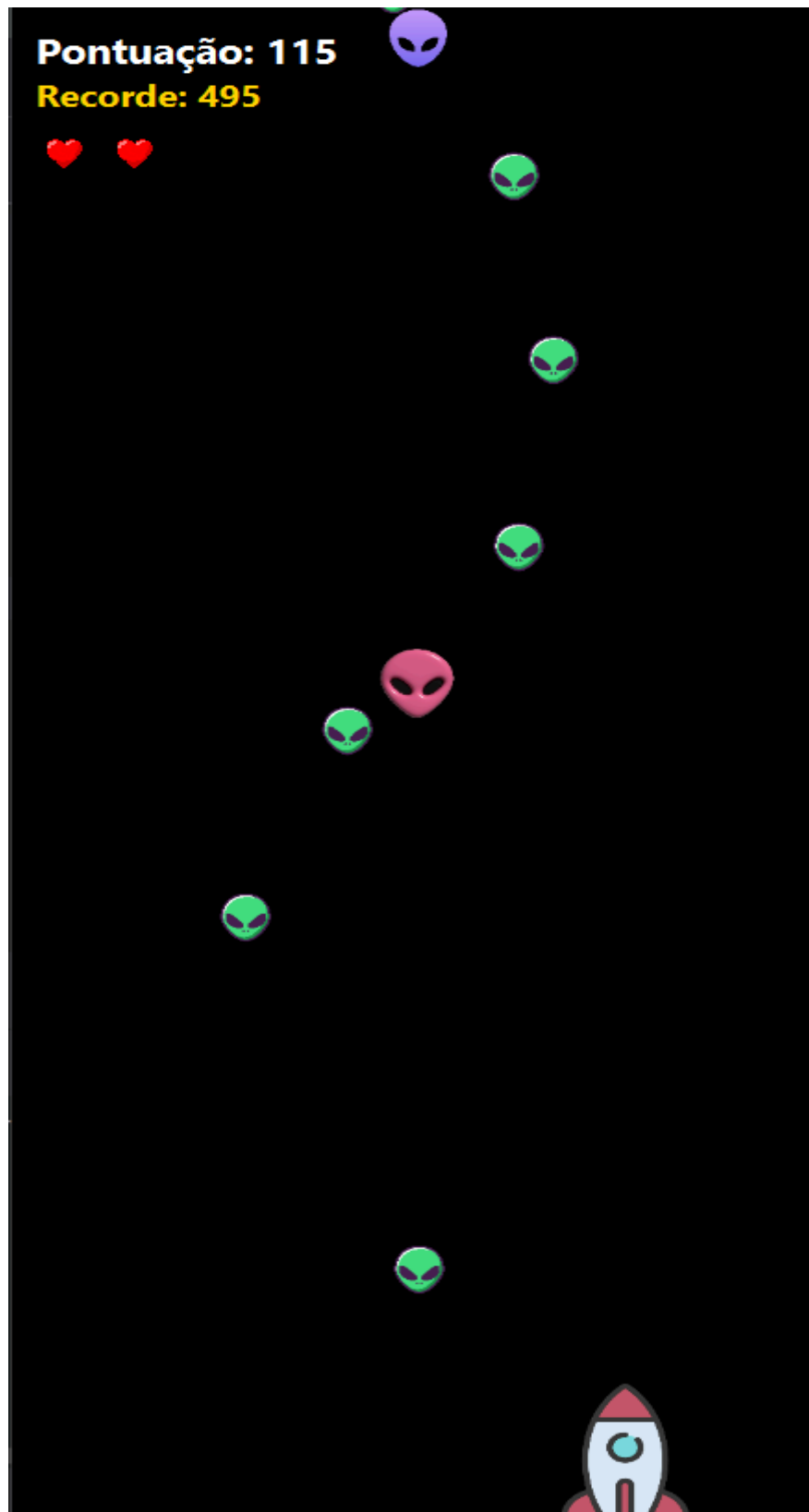
Esboço das Janelas (+ Funcionalidades Implementadas)

1. Ecrã de Configuração (Dificuldade):

- **Seleção de Nível:** Botões “Fácil” e “Difícil” para a seleção de níveis. No nível fácil, o jogador tem que conseguir ganhar 5000 pontos. No nível difícil, tem que ganhar 15000. Aparece um *checkmark* nesses botões se o jogador alguma vez conseguiu ganhar um desses níveis.
- **Toggle/Botão "Endless":** Ativa o modo endless. Em cima da seleção dos níveis, aparece um texto que apresenta o recorde no modo endless



2. **Ecrã de Jogo:** A interface JavaFX onde a ação ocorre. O jogador consegue mover-se da esquerda para a direita usando “A” e “D”. Para lançar projéteis, o jogador usa “SPACEBAR”. Cada inimigo vale 50 pontos. Os power-ups oferecem ao jogador 150 pontos.



3. **Ecrã de Fim de Jogo:** O jogador é introduzido a este ecrã sempre que perde o jogo. O botão “Tente Outra Vez” reinicia o jogo, enquanto que o botão “Sair...” leva o jogador de volta ao menu.



5. Testes Unitários

Devido às semelhanças entre todas as subclasses, não foram muitos os testes unitários que foram feitos. Porém, eis aqueles que foram criados para cada subclasse:

- **void getWidthReturnsWidth():** deve devolver o respectivo width da subclasse;
- **void getHeightReturnsHeight():** deve devolver o respectivo height da subclasse;
- **void isDeadReturnsFalse():** deve devolver falso porque é esse o valor definido por definição;

Antes de cada teste (@BeforeEach) é criado um objeto para cada subclasse, para ele ser reutilizado em cada teste dessa subclasse.

6. Tratamento de exceções

Apesar de não terem aparecido exceções no desenvolvimento do programa, houve certos momentos em que, num certo método, foi preciso fazer tratamento de exceções. Foram, assim, criadas duas exceções personalizadas:

- **PontuacaoInvalidaException:** permite avaliar se a pontuação guardada e carregada no modo endless é correta ou não;
- **RecursoInvalidoException:** permite avaliar se uma certa imagem é carregada no jogo ou não

7. Dificuldades sentidas

A parte mais difícil deste trabalho foi certamente a criação da classe GestorJogo. É preciso criar um número elevado de métodos, sendo em que alguns deles são precisas contas matemáticas para manipular como as entidades móveis serão inseridas no jogo. O método mais difícil de implementar foi o “void backMenu()” pois foi requerido muitos atributos (booleanos e mais) para realizar a ação tão simples que é a mudança de cenário. Também foi preciso, de um modo geral, pesquisar como certas classes funcionam para implementar a interface gráfica, que não foram dadas em aula. O exemplo mais claro foi a classe GraphicContext (classe que renderiza um Node chamado Canva na interfaces).

8. Conclusão

E assim foi criado o jogo “Space Invaders: Evolution”, um jogo simples de matar aliens antes que eles cheguem ao chão, mas que permite vários modos de jogo. Para a criação desse jogo, foi preciso criar uma superclasse abstrata *EntidadeMovel* para representar os objetos no jogo e subclasses que herdaram desta (duas destas subclasses iram herdar de uma outra subclasse). Para além disso, foram necessárias implementações para construir a interface gráfica com JavaFX, criando botões, música, *sound effects*, criação e mudança de scenes, e uns 14 ou 15 métodos exteriores. Mas com encapsulamento e uso de “*boolean flags*”, o jogo conseguiu ser feito!

Muito obrigado por ter lido até aqui :)